

HuBMAP - Hacking the Human Vasculature

535521 Selected Topics in Visual Recognition using Deep Learning

徐葆驊、吳念澤、江怜儀、王好瑄

Group14

GitHub: https://github.com/benhstu0828/2026version_HuBMAP_Hacking_the_Human_Vasculature.git

1 Introduction

This project studies the Kaggle competition *HuBMAP – Hacking the Human Vasculature* [?]. The task is instance segmentation: for each 512×512 histology tile, the model must output separate object masks with confidence scores. The final evaluation follows a mean Average Precision (mAP) style metric over instance masks, so both localization quality and mask quality matter.

The training data is heterogeneous. Dataset 1 contains the cleanest annotations and is used as the main validation and fine-tuning source in our experiments. Dataset 2 contains more weak/noisy annotations and is useful for increasing data coverage, but it can also inject label noise. Dataset 3 provides additional whole-slide image (WSI) regions; in our implementation it is treated mainly as extra WSI context in the dataset metadata, while the main training pipelines focus on Dataset 1 and Dataset 2. This setting makes cross-validation design important: neighboring tiles from the same WSI can be visually similar, so random tile-level splits would overestimate generalization.

2 Top Solutions Analysis

We do not attempt to survey all leaderboard solutions. Instead, we focus on two ideas that directly influenced our implementation.

2.1 First-place direction: WSI-aware heterogeneous ensembles

The public first-place release is not a single model. In the released notebook and configs, the final system is a heterogeneous MMDetection ensemble that includes RTMDet-X, YOLOX-X, and Swin-based Mask R-CNN / RTMDet variants. The strongest single model in the release is an RTMDet-X variant with an extra FCN mask head. Its backbone is CSPNeXt, its neck is CSPNeXt-PAFPN, and the custom mask branch applies RoIAlign on multi-scale features before predicting binary instance masks.

Several design choices are especially relevant:

- **WSI-aware validation.** The released data preparation uses WSI/spatial splits such as `dtrain0i.json/dval0i.json`, avoiding overly optimistic random splits.
- **Strong geometric and color augmentation.** The configs use rotations, scale/crop transforms, horizontal flipping, and HSV/color augmentations. These act as stain- and morphology-robustness mechanisms without explicitly doing stain normalization.
- **Model diversity.** The final prediction is built from multiple detector families rather than a single architecture. WBF is used to fuse bounding boxes across streams.
- **Box-first mask refinement.** The final ensemble first merges boxes, then re-predicts masks from a strong Mask R-CNN branch. This is an important observation: mask mAP is strongly limited by box quality.

This project therefore uses the first-place pipeline as the main baseline: reproduce the MMDetection ensemble behavior, then test whether a new stream improves or hurts the fused result.

2.2 Third-place direction: multi-stage training

The other useful idea is the multi-stage training strategy: pretrain on noisy/larger data, then fine-tune on clean/smaller data. In our code this appears in `mmnet_v3_solution`: Stage 1 trains on Dataset 2, and Stage 2 fine-tunes on Dataset 1. This strategy is reasonable because Dataset 2 can teach broad morphological priors, while Dataset 1 should correct boundary precision and class noise.

However, the original high-ranking multi-stage systems used large high-capacity backbones and high-resolution training. Our experiment replaces that family with an InternImage-T backbone inside Cascade Mask R-CNN. This makes the experiment informative but not a faithful reproduction of the third-place architecture.

3 Baseline Reproduction

3.1 Baseline A: first-place MMDetection ensemble

The reproduced baseline follows the public first-place release. Its inference notebook runs ten MMDetection streams:

$$\{r0i, r1i, s0i, s1i, m0i, m1i, y0i, y1i, sb0i, sb1i\}.$$

Each stream outputs bounding boxes, scores, and labels. These boxes are fused by Weighted Boxes Fusion (WBF) [?]. The fused boxes are then passed into a Mask R-CNN style mask branch from `m1i` to produce final instance masks. The local workflow notes record this legacy MMDetection baseline as reproducing a Kaggle score of about 0.589.

3.2 Data processing

For MMDetection experiments, annotations are converted from `polygons.jsonl` to COCO format. Dataset 1 is split spatially within WSI 1 and WSI 2 to create `dtrain0i.json` and `dval0i.json`. Dataset 2 is exported as `dtrain_dataset2.json` for pre-training.

For YOLO experiments, we use a separate preprocessing pipeline. Each 512×512 tile is padded to 768×768 by stitching neighboring tiles when available. Neighbor polygons are shifted into the padded coordinate frame, which reduces truncation artifacts for vessels crossing tile boundaries. The output is written in Ultralytics YOLO segmentation format with four spatial folds. The split uses Dataset 1 for validation and excludes spatially overlapping Dataset 2 tiles from the corresponding training fold to reduce leakage.

4 Experiment 1: YOLO26-Seg Ensemble

4.1 Motivation

The first experiment asks whether adding a modern YOLO segmentation model can improve the first-place ensemble. The hypothesis is straightforward: YOLO26-Seg is architecturally different from Cascade/Mask R-CNN style models, so it may add useful diversity. If its errors are complementary, WBF should produce better boxes and therefore better final masks.

4.2 Method

We trained a four-fold YOLO26x-Seg ensemble using Ultralytics. The training script uses:

- input size 768×768 , matching the padded tile size;
- AdamW with fixed learning rate 10^{-3} ;
- heavy geometric augmentation: 180° rotation, flips, scale, mosaic, mixup, and copy-paste;
- class filtering at inference so that only `blood_vessel` instances are submitted.

The YOLO stream is inserted into the original notebook between the MMDetection inference cells and the WBF cell. Each YOLO fold produces `yolo0.pkl` to `yolo3.pkl`, using the same dictionary format as MMDetection:

$$\text{pred_instances} = \{\text{bboxes}, \text{scores}, \text{labels}\}.$$

This makes YOLO a drop-in WBF stream. In the modified WBF cell, the four YOLO streams are assigned weights $[2, 2, 2, 2]$ and can be enabled with `ABLATION_MODE = baseline`.

4.3 Results and analysis

The YOLO-only validation results were reasonable for a lightweight independent stream, but they did not beat the reproduced first-place ensemble. The best local fold-level YOLO mask `mAP@50:95` ranged from 0.137 to 0.215, as shown in Table ???. The Kaggle notebook notes also indicate that the YOLO-augmented WBF submission was lower than the legacy MMDetection-only baseline.

Fold	Best epoch	Mask mAP@50	Mask mAP@50:95
0	76	0.226	0.137
1	59	0.255	0.160
2	76	0.305	0.215
3	59	0.302	0.203

Table 1: Best local validation metrics of the YOLO26x-Seg folds from `runs/seg/fold*/results.csv`.

The negative result is still useful. First, YOLO did not provide enough diversity: many mistakes overlapped with the existing detector ensemble, especially on ambiguous vessel-like structures. Second, its bounding boxes were not consistently better than the Cascade/Mask R-CNN family. Since the final pipeline re-predicts masks from fused boxes, box quality dominates the final mAP. Third, YOLO masks can be visually sharp after padding, but better masks do not help if the corresponding boxes do not improve WBF clusters. Therefore, the experiment suggests that adding a segmentation stream is not sufficient; the stream must improve localization diversity.

5 Experiment 2: Two-stage InternImage Cascade

5.1 Motivation

The second experiment follows the multi-stage idea from high-ranking solutions: first learn from the larger noisy Dataset 2, then fine-tune on the cleaner Dataset 1. Instead of reproducing the original large ViT/CBNet/DetectoRS-style system, we implemented a more practical MMDetection v3 model that can run in our environment and be packaged for Kaggle.

5.2 Method

The model is Cascade Mask R-CNN [?] with:

- **Backbone:** InternImage-T with pure PyTorch DCNv3 operators;
- **Neck:** FPN for multi-scale feature fusion;
- **Head:** 3-stage Cascade RoI head with progressively stricter IoU thresholds;
- **Mask branch:** FCN mask head with mask loss weight increased to 2.0;
- **Resolution:** 1024×1024 training images.

Stage 1 trains on `dtrain_dataset2.json` for 10 epochs with light augmentation and AdamW learning rate 2×10^{-4} . Stage 2 loads the best Stage 1 checkpoint and fine-tunes on `dtrain01.json` for 20 epochs with stronger augmentation, including random flips, AutoAugment color/geometric policies, and Albumentations elastic transform. The Kaggle inference package uses FP16, six-view TTA, morphological post-processing, and offline MMDetection dependencies.

5.3 Results and analysis

Stage 1 reached local `coco/segm_mAP` 0.130 at epoch 8 on `dval01`. After fine-tuning on Dataset 1, Stage 2 improved substantially and reached 0.440 local `coco/segm_mAP` at epoch 12. This confirms that the Dataset2 $\rightarrow$ Dataset1 schedule is useful. However, the result still did not surpass the reproduced first-place ensemble, and it is below what one would expect from a faithful third-place-style system.

Model / stage	Data	Best epoch	BBox mAP	Segm mAP
InternImage Cascade Stage 1	Dataset 2	8	0.181	0.130
InternImage Cascade Stage 2	Dataset 1	12	0.407	0.440

Table 2: Local MMDetection validation metrics on `dval01`. These are not directly comparable to Kaggle private leaderboard scores.

The main reason is architecture mismatch. The referenced multi-stage solution direction relies on very large transformer-style backbones and detector ensembles. Our InternImage-T model is much smaller and is a different system. A second reason is Dataset 2 noise:

a compact model may absorb noisy masks more directly, whereas a larger pretrained backbone can be more robust. Finally, our implementation used only a single InternImage Cascade stream. The first-place baseline derives much of its strength from ensembling and from using a dedicated high-resolution mask re-prediction stage.

6 Discussion

The two experiments show that leaderboard-level instance segmentation is a systems problem, not only a model-choice problem. The first-place pipeline works because it separates responsibilities: many models propose boxes, WBF filters and averages localization evidence, and a strong mask branch converts fused boxes into final masks. This structure also explains why the YOLO experiment did not help. YOLO26-Seg produced masks, but the ensemble bottleneck was box quality and localization diversity.

The InternImage experiment gives a different lesson. Multi-stage training is useful: Stage 2 improved the local segmentation mAP from 0.130 to 0.440. But borrowing only the training schedule is not enough to reproduce a top solution. Backbone capacity, resolution, label-noise robustness, and ensemble integration all interact. InternImage-T was attractive because it was easier to package and run, especially with a pure PyTorch DCNv3 implementation, but this practical choice also limited its ceiling.

Overall, our best direction remains the reproduced first-place MMDetection ensemble. The YOLO and InternImage experiments are valuable ablations: YOLO tested whether simple diversity improves WBF, while InternImage tested whether Dataset2 pretraining transfers to Dataset1 fine-tuning. Both experiments clarify where future improvements should focus: better box proposals, higher-capacity backbones, stronger WSI-aware validation, and more careful fusion rather than simply adding more segmentation outputs.

7 Conclusion

We reproduced the structure of the HuBMAP first-place solution and explored two extensions. Adding YOLO26x-Seg as a four-fold WBF stream increased system diversity but did not improve the final result, mainly because it did not improve fused bounding boxes. A two-stage Cascade Mask R-CNN with InternImage-T confirmed the benefit of Dataset2 pretraining followed by Dataset1 fine-tuning, but the smaller backbone and single-model setup remained below the reproduced ensemble baseline. The project therefore supports a central conclusion: for this competition, high-quality box localization and ensemble design are more important than mask sharpness alone.

References

- [1] Vaishnav Potlapalli, Syed Waqas Zamir, Salman H. Khan, and Fahad Shahbaz Khan. Promptir: Prompting for all-in-one image restoration. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [2] Syed Waqas Zamir, Aditya Arora, Salman Khan, Munawar Hayat, Fahad Shahbaz Khan, and Ming-Hsuan Yang. Restormer: Efficient transformer for high-resolution image restoration. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 5728–5739, 2022.
- [3] Yuning Cui, Wenqi Ren, Xiaochun Cao, and Alois Knoll. AdaIR: Adaptive all-in-one image restoration via frequency mining and modulation. In *International Conference on Learning Representations (ICLR)*, 2024.
- [4] Lingshun Kong, Jiangxin Dong, Jianjun Ge, Mingqiang Li, and Jinshan Pan. Efficient frequency domain-based transformers for high-quality image deblurring. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [5] Jie Hu, Li Shen, and Gang Sun. Squeeze-and-excitation networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 7132–7141, 2018.
- [6] Towaki Takikawa, David Acuna, Varun Jampani, and Sanja Fidler. Gated-SCNN: Gated shape CNNs for semantic segmentation. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019.
- [7] Pierre Charbonnier, Laure Blanc-Feraud, Gilles Aubert, and Michel Barlaud. Two deterministic half-quadratic regularization algorithms for computed imaging. In *Proceedings of the IEEE International Conference on Image Processing (ICIP)*, volume 2, pages 168–172, 1994.
- [8] Zhou Wang, Alan C. Bovik, Hamid R. Sheikh, and Eero P. Simoncelli. Image quality assessment: From error visibility to structural similarity. *IEEE Transactions on Image Processing*, 13(4):600–612, 2004.
- [9] Antti Tarvainen and Harri Valpola. Mean teachers are better role models: Weight-averaged consistency targets improve semi-supervised deep learning results. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

- [10] Bee Lim, Sanghyun Son, Heewon Kim, Seungjun Nah, and Kyoung Mu Lee. Enhanced deep residual networks for single image super-resolution. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, pages 136–144, 2017.